

Лабораторная работа №2

Основы вероятностного моделирования угроз

Бабенко Артём Сергеевич

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
5	Контрольные вопросы	35
6	Выводы	39
	Список литературы	40

Список иллюстраций

4.1	Скрипт src/simulation.jl	8
4.2	Скрипт scripts/run_experiment.jl	9
4.3	Скрипт scripts/analyze.jl	10
4.4	Скрипт scripts/convergence.jl	10
4.5	Скрипт scripts/parameter_sweep.jl	11
4.6	Детальные графики	12
4.7	Детальные графики	12
4.8	Детальные графики	13
4.9	Детальные графики	13
4.10	Детальные графики	14
4.11	Формат qmd	14
4.12	Формат irunb	15
4.13	Скрипт для генерации производных форматов	15

1 Цель работы

Освоить базовые методы вероятностного моделирования случайных процессов в контексте кибербезопасности.

На примере моделирования потока атак на веб-сервер изучить:

- генерацию случайных величин с заданным распределением;
- статистический анализ смоделированных данных;
- проверку соответствия эмпирического распределения теоретическому;
- оценку вероятностей редких событий;
- определить набор параметров модели (интенсивность атак, длительность наблюдения и т.д.);
- реализовать симуляцию пуассоновского потока атак;
- выполнить статистический анализ результатов;
- визуализировать данные и проверить соответствие теоретическим распределениям.

2 Задание

1. Создать проект DrWatson с именем project.
2. Определить набор параметров модели (словарь params) с возможностью варьирования.
3. Написать функцию simulate_attacks(params), которая возвращает структуру с результатами симуляции (например, массив числа атак по часам, интервалы, моменты времени).
4. Использовать produce_or_load для запуска симуляции с заданными параметрами и сохранения результатов на диск.
5. Провести серию экспериментов с разными значениями параметров (например, разные λ , разная длительность).
6. Загрузить сохранённые результаты и выполнить:
 - построение гистограммы числа атак за час и сравнение с теоретическим распределением Пуассона;
 - построение графика накопленного числа атак и интервалов между атаками;
 - проверку экспоненциальности интервалов (гистограмма, QQ-plot, возможно критерий согласия);
 - оценку вероятности события «более 10 атак за час» теоретически и эмпирически.
7. Проанализировать зависимость точности оценки вероятности от числа симуляций.

3 Теоретическое введение

Во многих задачах кибербезопасности поток атак (или инцидентов) можно приближённо считать простейшим (пуассоновским) потоком. Для такого потока число событий N_t за время t распределено по закону Пуассона:

$$P(N_t = k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad k = 0, 1, 2, \dots$$

где λ — интенсивность потока (среднее число событий в единицу времени).

В простейшем потоке интервалы времени между соседними событиями $\tau_1, \tau_2, \dots$ независимы и имеют экспоненциальное распределение с параметром λ :

$$f(\tau) = \lambda e^{-\lambda \tau}, \quad \tau \geq 0$$

Среднее значение интервала: $\mathbb{E}[\tau] = 1/\lambda$.

4 Выполнение лабораторной работы

Представленные скрипты образуют исследовательский конвейер:

- Моделирование (`simulation.jl`) — вычислительное ядро.
- Генерация данных (`run_experiment.jl`) — получение реализаций для фиксированных параметров.
- Анализ и визуализация (`analyze.jl`) — проверка соответствия модели теоретическим распределениям.
- Исследование сходимости (`convergence.jl`) — изучение точности оценок.
- Параметрическое исследование (`parameter_sweep.jl`) — анализ чувствительности к изменению интенсивности.

Для выполнения работы необходимо создать проект с помощью DrWatson. Затем создадим в этом проекте скрипт `src/simulation.jl`, который содержит функцию `simulate_attacks`, реализующую вероятностную модель потока атак (рис. 4.1).

```

simulation.jl
using Distributions
using Statistics

function simulate_attacks(lmb::Float64, T::Float64)
    hourly_counts = rand(Poisson(lmb), floor(Int, T)) # Моделирование числа атак по часам (целые часы)
    intervals = Float64[] # Моделирование интервалов между атаками (точные моменты)
    total_time = 0.0
    while total_time < T
        τ = rand(Exponential(1/lmb))
        push!(intervals, τ)
        total_time += τ
    end

    # Удаляем последнее событие, если оно вышло за пределы T
    if total_time > T
        pop!(intervals)
    end
    attack_times = cumsum(intervals)
    return (hourly_counts = hourly_counts, intervals = intervals, attack_times = attack_times)
end

# Обёртка для вызова из скрипта с параметром-словарём
function simulate_attacks(p::Dict)
    return simulate_attacks(p[:lmb], p[:T])
end

```

Рисунок 4.1: Скрипт src/simulation.jl

Затем создадим скрипт scripts/run_experiment.jl. Он выполняет симуляцию с заданными параметрами, вычисляет эмпирическую вероятность события «более 10 атак за час» и сохраняет все результаты на диск (рис. 4.2). Скрипт предназначен для первичной генерации данных, которые потом будут анализироваться в analyze.jl.

```

> run_experiment.jl
using DrWatson
@quickactivate "project"
using Distributions
using Statistics
using JLD2

include(scrdir("simulation.jl"))

params = Dict(
    :λ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)

function run_simulation(p)
    @unpack λ, T, num_hours_for_est = p
    res = simulate_attacks(λ, T)
    hourly_sample = rand(Poisson(λ), num_hours_for_est)
    emp_prob = count(hourly_sample .> 10) / num_hours_for_est
    theor_prob = 1 - cdf(Poisson(λ), 10)
    return Dict(
        :hourly_counts => res.hourly_counts,
        :intervals => res.intervals,
        :attack_times => res.attack_times,
        :emp_prob => emp_prob,
    )
end

```

Рисунок 4.2: Скрипт scripts/run_experiment.jl

Теперь создадим скрипт scripts/analyze.jl. Он загружает сохранённые данные и строит четыре диагностических графика, позволяющих визуально оценить соответствие смоделированного потока теоретическим предположениям (пуассоновости и экспоненциальности) (рис. 4.3).

```

> analyze.jl
using DrWatson
@quickactivate "project"
using Plots, Distributions, StatsPlots, JLD2

# Определяем параметры эксперимента (должны совпадать с использованными в run_experiment.jl)

params = Dict{
    :λ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
}

# Формируем путь к файлу с данными (как в run_experiment.jl)
filename = datadir("attack_sim", savename(params, "jld2"))
# Проверим существование файла
if !isfile(filename)
    error("Файл $filename не найден. Сначала запустите scripts/run_experiment.jl для генерации данных.")
end

# Загружаем данные
@load filename data
# data – это словарь (Dict), сохранённый в run_experiment.jl
# Извлекаем результаты
hourly_counts = data[:hourly_counts]
intervals = data[:intervals]

```

Рисунок 4.3: Скрипт scripts/analyze.jl

Также добавим скрипт scripts/convergence.jl (рис. 4.4). Он позволяет изучить, как быстро эмпирическая оценка вероятности редкого события приближается к теоретическому значению при увеличении объёма выборки.

```

> convergence.jl
using DrWatson
@quickactivate "project"
using Distributions, Plots, Statistics, Random, JLD2

# Параметры модели
λ = 5.0
theor_prob = 1 - cdf(Poisson(λ), 10)

# Размеры выборки для исследования (логарифмическая шкала)
sample_sizes = [10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]

# Для воспроизводимости
Random.seed!(123)

# Вычисляем эмпирические оценки для каждого объёма выборки
estimates = Float64[]
println("Вычисление оценок вероятности...")
for n in sample_sizes
    hourly_sample = rand(Poisson(λ), n)
    emp_prob = count(hourly_sample .> 10) / n
    push!(estimates, emp_prob)
    println("n = $n: оценка = $emp_prob")
end

# Построение графика сходимости
p = plot(sample_sizes, estimates,
    xscale = :log10,
    marker = :circle

```

Рисунок 4.4: Скрипт scripts/convergence.jl

Создадим скрипт `scripts/parameter_sweep.jl` (рис. 4.5). Позволяет систематически изучить, как изменение интенсивности атак влияет на характеристики потока и, в частности, на вероятность $P(>10)$.

```
> 📄 parameter_sweep.jl
using DrWatson
@quickactivate "project"
using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV, DataFrames

# Включаем функцию симуляции
include(sourcedir("simulation.jl"))

# Базовые параметры (общие для всех запусков)
base_params = Dict{
    :T => 24.0,
    :num_hours_for_est => 10000
}

# Диапазон значений λ для исследования
λ_values = [2.0, 5.0, 8.0, 12.0, 15.0]

# Для воспроизводимости
Random.seed!(42)

# Создаём папку для графиков параметрического исследования, если её нет
parametric_plots_dir = plotsdir("parameter_sweep")
mkpath(parametric_plots_dir)

# Структура для хранения сводных результатов
summary = Dict{Float64, Dict}{}
println("Запуск параметрического исследования...")

for λ in λ_values
    # Формируем полный словарь параметров
    params = merge(base_params, Dict{<math>\lambda</math> => λ})
```

Рисунок 4.5: Скрипт `scripts/parameter_sweep.jl`

Детальные графики для каждого λ в `plots/parameter_sweep/`.

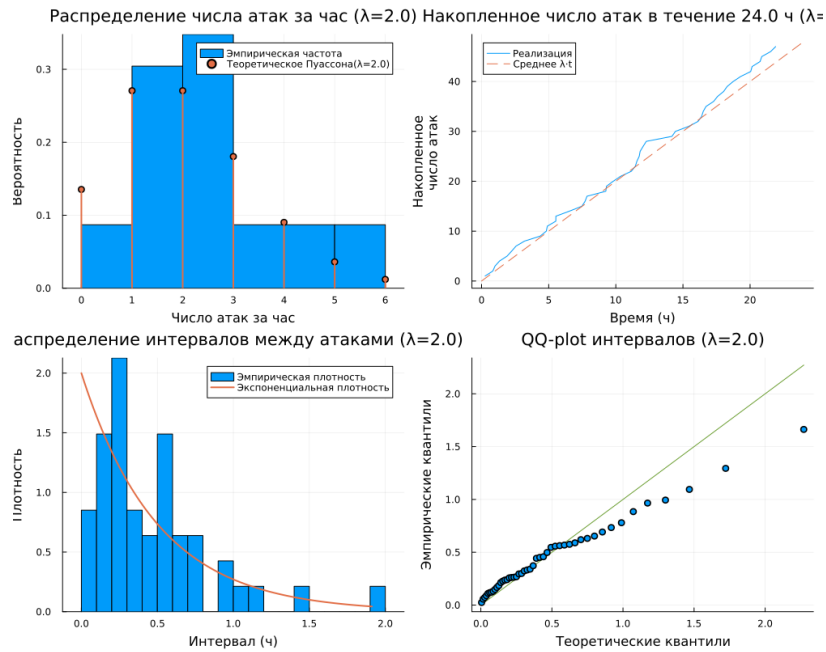


Рисунок 4.6: Детальные графики

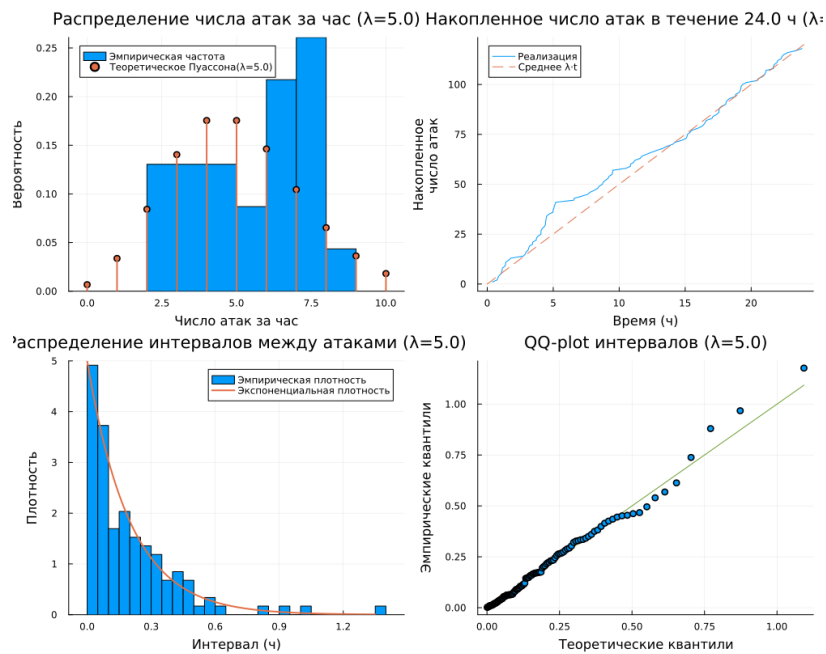


Рисунок 4.7: Детальные графики

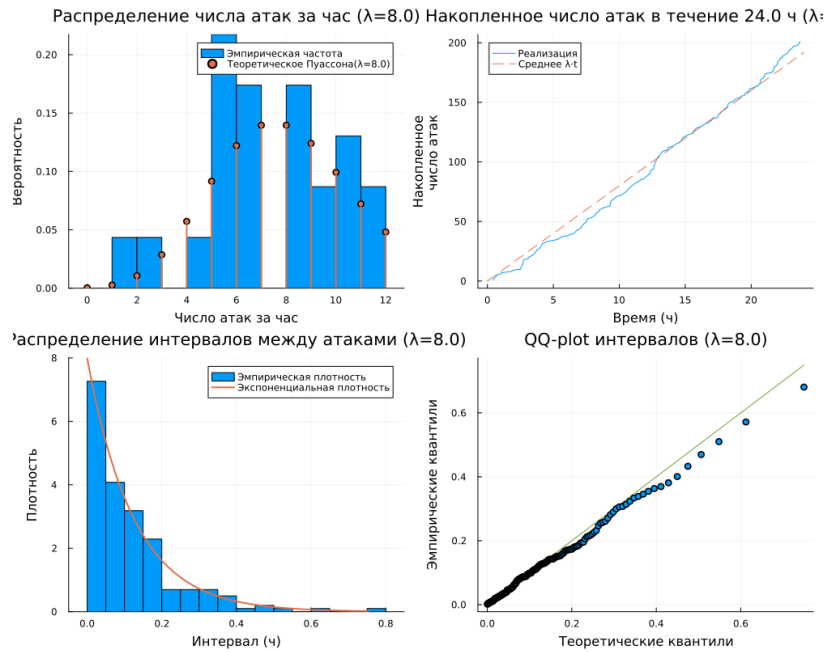


Рисунок 4.8: Детальные графики

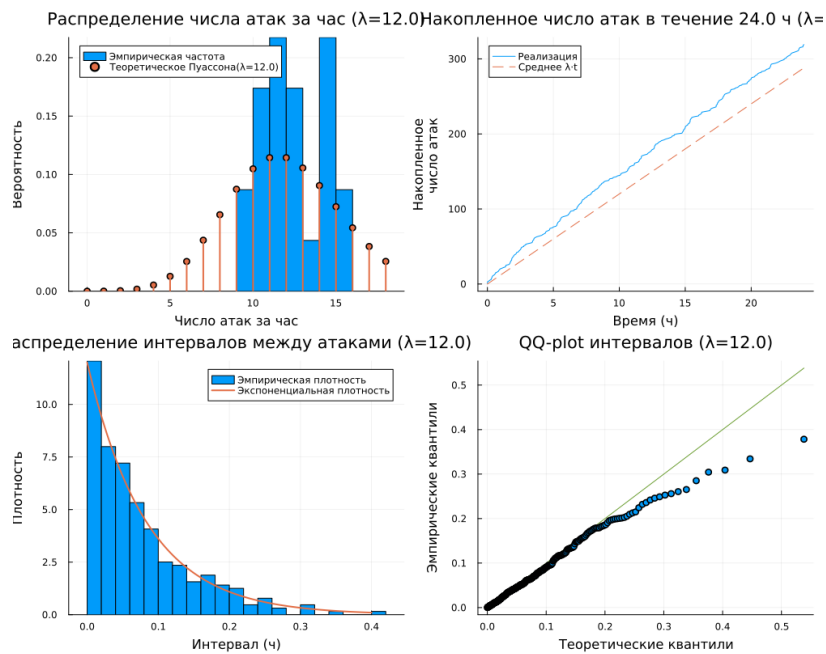


Рисунок 4.9: Детальные графики

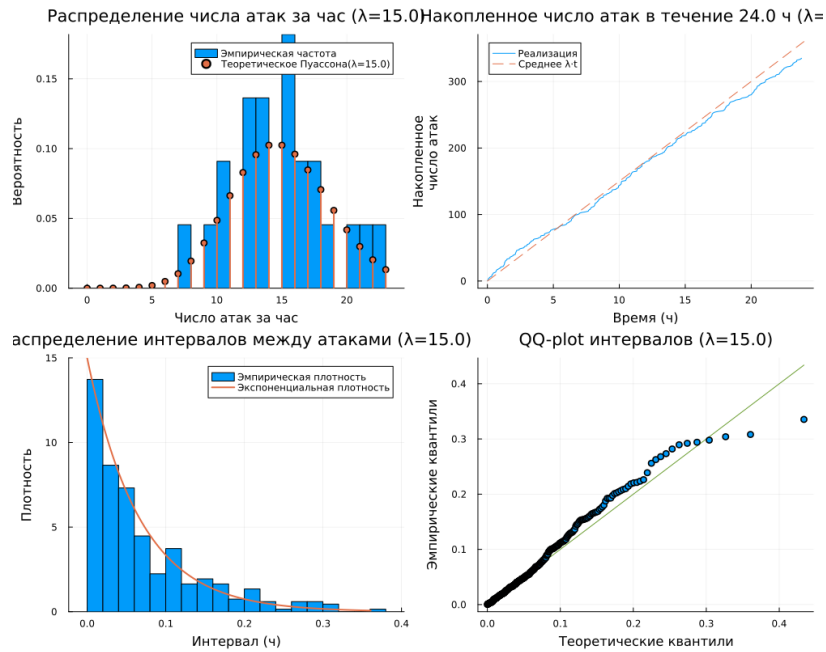


Рисунок 4.10: Детальные графики

Создадим скрипт для генерации производных форматов `scripts/tangle.jl` (рис. 4.13).
В результате получим код, оформленный в разных форматах (рис. 4.11, рис. 4.12).

```

>Run Cell | Run Next Cell
```{julia}
using DrWatson
@quickactivate "project"
using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV, DataFrames
```

Включаем функцию симуляции

>Run Cell | Run Next Cell | Run Above
```{julia}
include(scrdir("simulation.jl"))
```

Базовые параметры (общие для всех запусков)

>Run Cell | Run Next Cell | Run Above
```{julia}
base_params = Dict{
 :T => 24.0,
 :num_hours_for_est => 10000
}
```

Диапазон значений  $\lambda$  для исследования

>Run Cell | Run Next Cell | Run Above
```{julia}
lambda_values = [2.0, 5.0, 8.0, 12.0, 15.0]

```

Рисунок 4.11: Формат qmd

```

> using DrWatson
@quickactivate "project"
using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV, DataFrames

[]

Включаем функцию симуляции

include(srcdir("simulation.jl"))

[]

Базовые параметры (общие для всех запусков)

base_params = Dict{
 :T => 24.0,
 :num_hours_for_est => 10000
}

[]

Диапазон значений λ для исследования

λ values = [2.0, 5.0, 8.0, 12.0, 15.0]

```

Рисунок 4.12: Формат `ipynb`

```

> tangle.jl
#!/usr/bin/env julia
tangle.jl - Генератор отчетов из Literate-скриптов
Использование: julia tangle.jl <путь_к_скрипту>

using DrWatson
@quickactivate # Активирует текущий проект DrWatson

using Literate
function main()
 if length(ARGS) == 0
 println("""
 Использование: julia tangle.jl <путь_к_скрипту>
 Примеры:
 julia tangle.jl scripts/lab1.jl
 """)
 return
 end
 script_path = ARGS[1]
 if !isfile(script_path)
 error("Файл не найден: $script_path")
 end
 # Пути и имена
 script_dir = dirname(script_path)
 script_name = splitext(basename(script_path))[1]
 println("Генерация из: $script_path")
 # Чистый скрипт (без комментариев)
 scripts_dir = scriptsdir(script_name)
 Literate.script(script_path, scripts_dir; credit=false)
 println("✓ Чистый скрипт: $(scripts_dir)/$(script_name).jl")
end

```

Рисунок 4.13: Скрипт для генерации производных форматов

```

using DrWatson
@quickactivate "project"
using Distributions
using Statistics
using JLD2

include(sourcedir("simulation.jl"))

params = Dict(
 :λ => 5.0,
 :T => 24.0,
 :num_hours_for_est => 10000
)

function run_simulation(p)
 @unpack λ, T, num_hours_for_est = p
 res = simulate_attacks(λ, T)
 hourly_sample = rand(Poisson(λ), num_hours_for_est)
 emp_prob = count(hourly_sample .> 10) / num_hours_for_est
 theor_prob = 1 - cdf(Poisson(λ), 10)
 return Dict(
 :hourly_counts => res.hourly_counts,
 :intervals => res.intervals,
 :attack_times => res.attack_times,
 :emp_prob => emp_prob,
 :theor_prob => theor_prob
)
end

```



```

julia> cd("C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est")
julia> P(>10) = 0.0153
julia> P(>10) = 0.013695268598382881

```

```

using DrWatson
@quickactivate "project"
using Plots, Distributions, StatsPlots, JLD2

```

```

└ Warning: DrWatson could not find find a project file by recursively c
└ (given dir: C:\Users\Admin)
└ @ DrWatson C:\Users\Admin\.julia\packages\DrWatson\2QF5p\src\proje

```

Определяем параметры эксперимента (должны совпадать с использованными в run\_experiment.jl)

```

params = Dict{
 :T => 5.0,
 :T => 24.0,
 :num_hours_for_est => 10000
}

```

Dict{Symbol, Real} with 3 entries:

```

:T => 24.0
:T => 5.0
:num_hours_for_est => 10000

```

Формируем путь к файлу с данными (как в run\_experiment.jl)

```
filename = datadir("attack_sim", savename(params, "jld2"))
```

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab02\\p
```

Проверяем существование файла

```
if !isfile(filename)
 error("Файл $filename не найден. Проверьте путь scripts/run_experiment.jl")
end
```

Загружаем данные

```
@load filename data
```

```
1-element Vector{Symbol}:
:data
```

data — это словарь (Dict), сохранённый в run\_experiment.jl Извлекаем результаты

```
hourly_counts = data[:hourly_counts]
intervals = data[:intervals]
attack_times = data[:attack_times]
X = params[:X]
T = params[:T]
emp_prob = data[:emp_prob]
theor_prob = data[:theor_prob]
```

```
0.013695268598382881
```

Выводим вероятности для справки

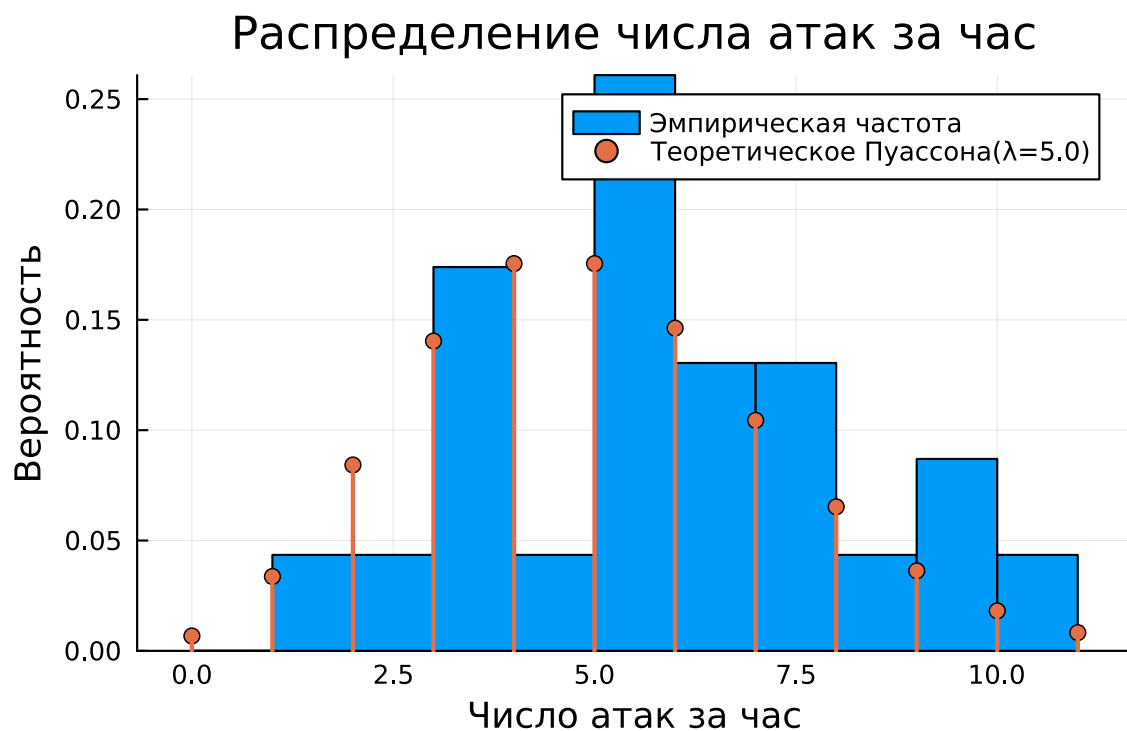
```
println("Эмпирическая вероятность P(>10) = ", emp_prob)
println("Теоретическая вероятность P(>10) = ", theor_prob)
```

Эмпирическая вероятность P(>10) = 0.0153

Теоретическая вероятность P(>10) = 0.013695268598382881

## 1. Гистограмма числа атак за час

```
p1 = histogram(hourly_counts, bins = 0:maximum(hourly_counts), normal = false,
 label = "Эмпирическая частота", xlabel = "Число атак за час", ylabel = "Вероятность")
x_vals = 0:maximum(hourly_counts)
theor_probs = pdf.(Poisson(5), x_vals)
plot!(p1, x_vals, theor_probs, line = :stem, marker = :circle, label = "Теоретическое Пуассона(λ=5.0)",
 title!(p1, "Распределение числа атак за час"))
```

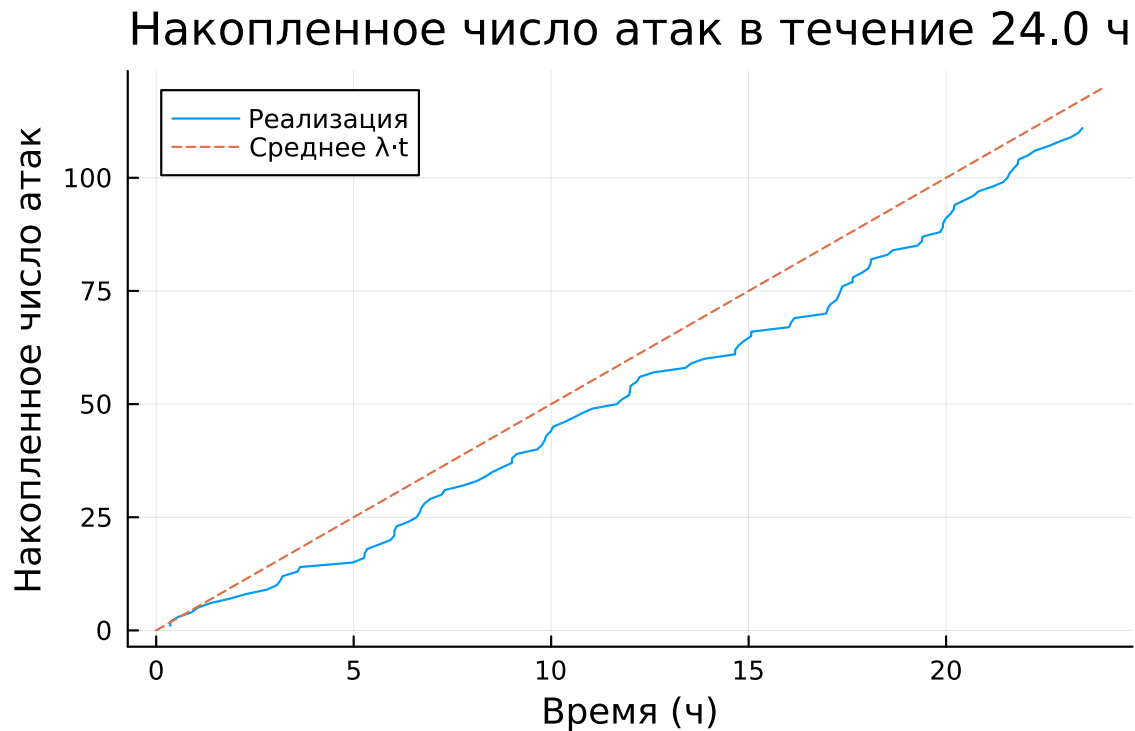


## 2. Накопленное число атак

```

p2 = plot(attack_times, 1:length(attack_times), label = "Накопленное число атак", x1
plot!(p2, 0:0.1:T, λ*(0:0.1:T), label = "Среднее λ·t", ls = :dash)
title!(p2, "Накопленное число атак в течение 24.0 ч")

```

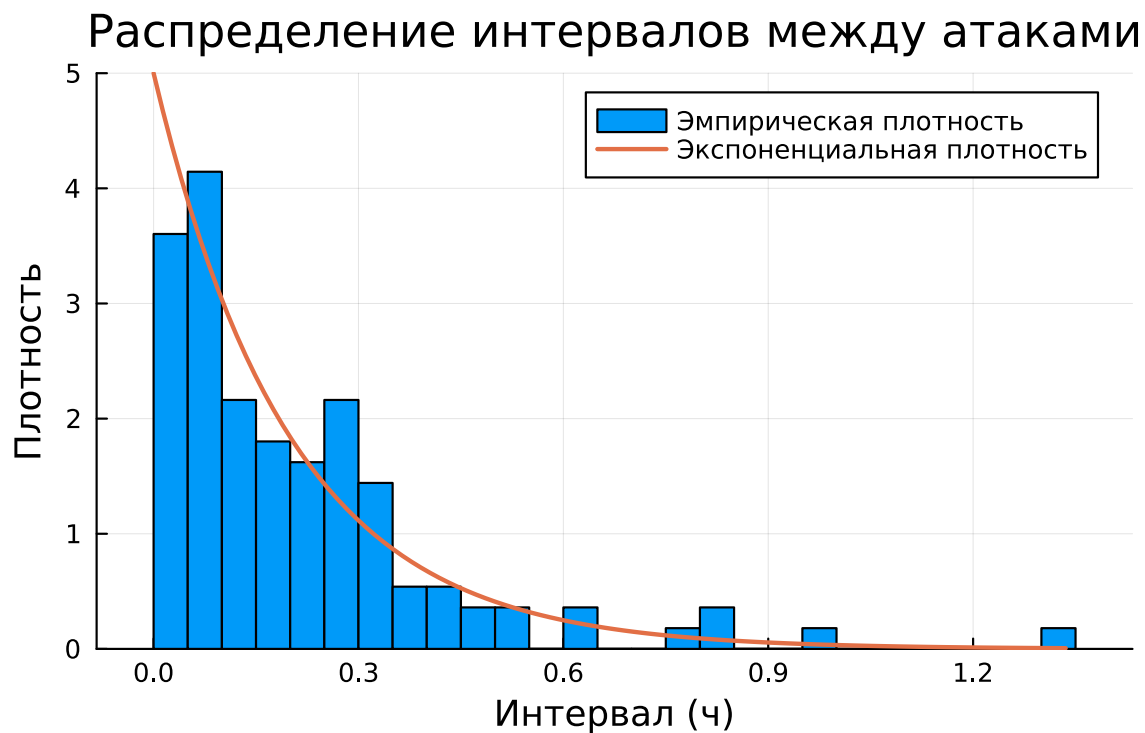


### 3. Гистограмма интервалов

```

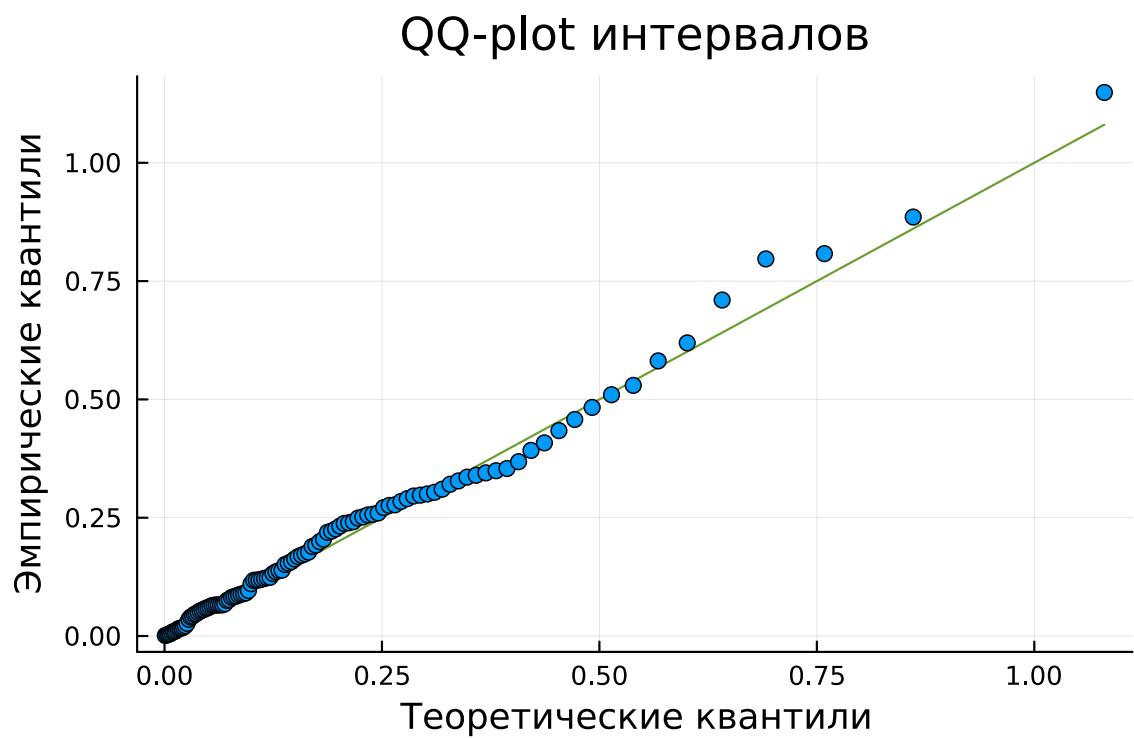
p3 = histogram(intervals, bins = 30, normalize = :pdf, label = "Гистограмма интервалов")
xlabel = "Интервал (ч)", ylabel = "Плотность")
x_dens = range(0, maximum(intervals), length=100)
theor_dens = pdf.(Exponential(1/λ), x_dens)
plot!(p3, x_dens, theor_dens, label = "Теоретическая плотность", lw=2)
title!(p3, "Гистограмма интервалов с теоретической плотностью")

```



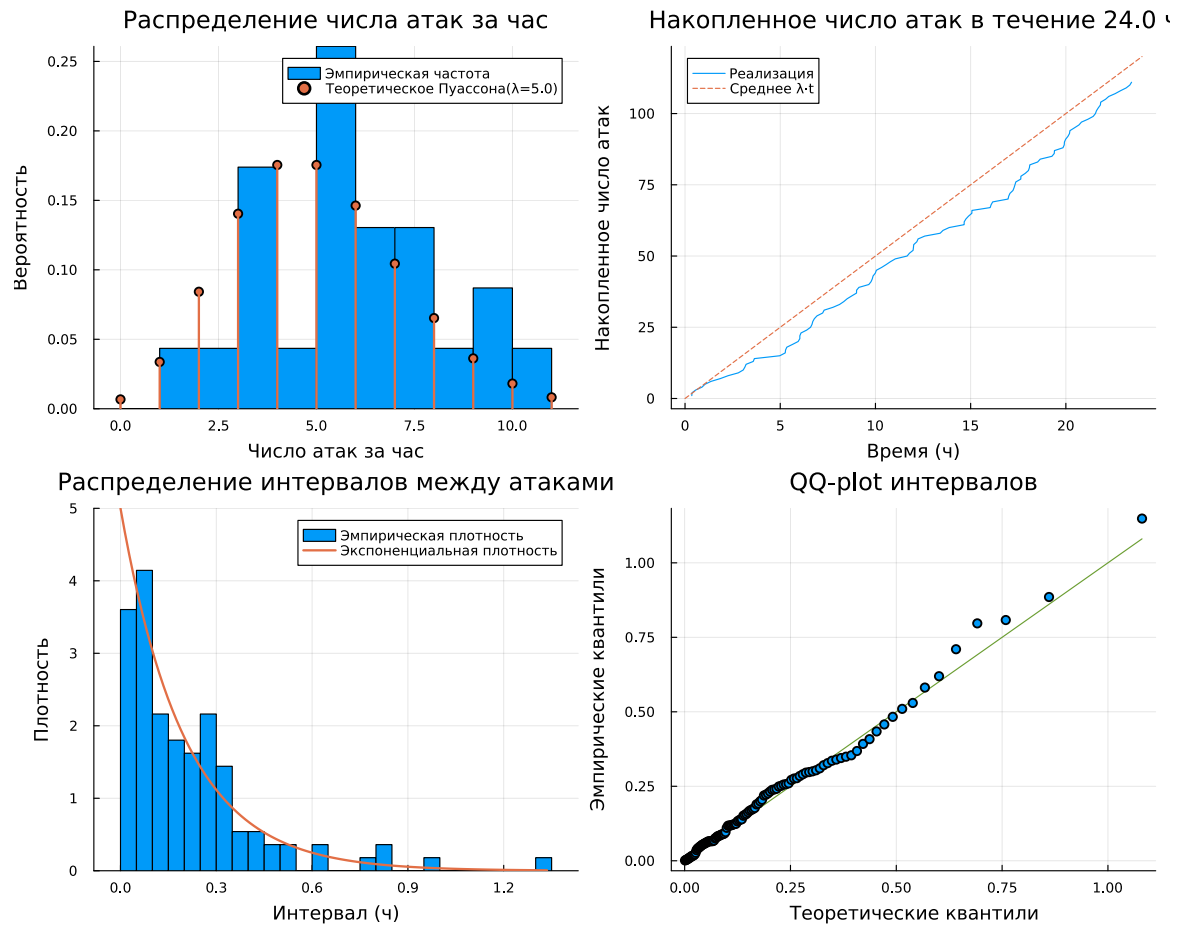
4. QQ-plot интервалов против экспоненциального распределения

```
p4 = qqplot(Exponential(1/λ), intervals, qqline = :identity,
 xlabel = "Экспоненциальное распределение", ylabel = "Эмпирическое распределение",
 title = "QQ-plot Эмпирическое распределение")
```



Объединяем графики

```
plot(p1, p2, p3, p4, layout = (2,2), size = (1000, 800))
```



Сохраняем графики в папку plots

```
savefig(plotsdir("attack_sim_plots.png"))
println("XXXXXXXX XXXXXXXXXXX X ", plotsdir("attack_sim_plots.png"))
```

```
XXXXXXXX XXXXXXXXXXX X C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\plots\attack_sim_plots.png
```

```
using DrWatson
@quickactivate "project"
using Distributions, Plots, Statistics, Random, JLD2
```

└ Warning: DrWatson could not find a project file by recursively cl

⊠ (given dir: C:\Users\Admin)

└ @ DrWatson C:\Users\Admin\.julia\packages\DrWatson\2QF5p\src\projec

Параметры модели

```
⊠ = 5.0
theor_prob = 1 - cdf(Poisson(⊠), 10)
```

0.013695268598382881

Размеры выборки для исследования (логарифмическая шкала)

```
sample_sizes = [10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]
```

9-element Vector{Int64}:

10  
50  
100  
500  
1000  
5000  
10000  
50000  
100000

Для воспроизводимости

```
Random.seed!(123)
```

TaskLocalRNG()

Вычисляем эмпирические оценки для каждого объёма выборки

```

estimates = Float64[]
println("XXXXXXXXXX XXXXXX XXXXXXXXXXXXX...")
for n in sample_sizes
 hourly_sample = rand(Poisson(0), n)
 emp_prob = count(hourly_sample .> 10) / n
 push!(estimates, emp_prob)
 println("n = $n: XXXXXX = $emp_prob")
end

```

XXXXXXXXXX XXXXXX XXXXXXXXXXXXX...

```

n = 10: XXXXXX = 0.0
n = 50: XXXXXX = 0.02
n = 100: XXXXXX = 0.01
n = 500: XXXXXX = 0.018
n = 1000: XXXXXX = 0.014
n = 5000: XXXXXX = 0.016
n = 10000: XXXXXX = 0.0131
n = 50000: XXXXXX = 0.0136
n = 100000: XXXXXX = 0.0139

```

Построение графика сходимости

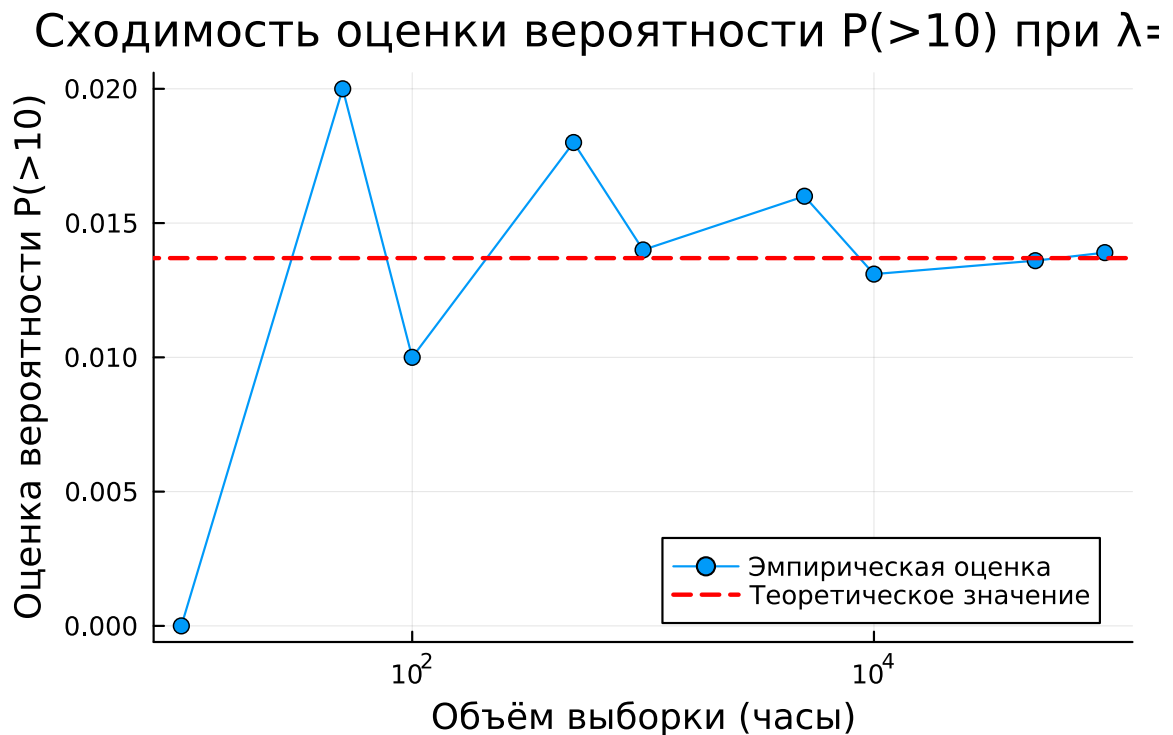
```

p = plot(sample_sizes, estimates,
 xscale = :log10,
 marker = :circle,
 label = "XXXXXXXXXXXXXXXX XXXXXX",
 xlabel = "XXXXXX XXXXXXXX (XXXX)",
 ylabel = "XXXXXX XXXXXXXXXXXXXXX P(>10)",
 legend = :bottomright)

```

```
hline!(p, [theor_prob],
 label = "Теоретическое значение",
 ls = :dash,
 lw = 2,
 color = :red)

title!(p, "Сходимость оценки вероятности P(>10) при λ=$λ")
```



Сохраняем график в папку plots

```
plot_path = plotsdir("convergence.png")
savefig(p, plot_path)
println("График сохранен в $plot_path")
```

Путь к файлу: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\plots\convergence.png

Сохраняем данные сходимости в JLD2 (опционально)

```
data_path = datadir("convergence", "convergence_data_@=$(id).jld2")
mkpath(datadir("convergence"))
@save data_path sample_sizes estimates @ theor_prob
println("Создан файл сходимости @ $(id) в $data_path")
```

```
@= C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\convergence\convergence_data_@=5.0
```

```
using DrWatson
@quickactivate "project"
using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV,
```

```
└ Warning: DrWatson could not find find a project file by recursively cl
@ (given dir: C:\Users\Admin)
└ @ DrWatson C:\Users\Admin\.julia\packages\DrWatson\2QF5p\src\projec
```

Включаем функцию симуляции

```
include(sourcedir("simulation.jl"))
```

simulate\_attacks (generic function with 2 methods)

Базовые параметры (общие для всех запусков)

```
base_params = Dict(
 :T => 24.0,
 :num_hours_for_est => 10000
)
```

Dict{Symbol, Real} with 2 entries:

```
:T => 24.0
:num_hours_for_est => 10000
```

Диапазон значений  $\lambda$  для исследования

```
λ_values = [2.0, 5.0, 8.0, 12.0, 15.0]
```

5-element Vector{Float64}:

```
2.0
5.0
8.0
12.0
15.0
```

Для воспроизводимости

```
Random.seed!(42)
```

TaskLocalRNG()

Создаём папку для графиков параметрического исследования, если её нет

```
parametric_plots_dir = plotsdir("parameter_sweep")
mkpath(parametric_plots_dir)
```

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab02\\p
```

Структура для хранения сводных результатов



```

intervals = data[:intervals]
attack_times = data[:attack_times]

p1 = histogram(hourly_counts, bins = 0:maximum(hourly_counts), norm
 label = "Histogram of hourly counts", xlabel = "Hourly counts", ylabel = "Frequency",
 x_vals = 0:maximum(hourly_counts)
 theor_probs = pdf.(Poisson(λ), x_vals)
plot!(p1, x_vals, theor_probs, line = :stem, marker = :circle, label = "Poisson fit")
title!(p1, "Histogram of hourly counts (lambda=$lambda)")

p2 = plot(attack_times, 1:length(attack_times), label = "Attack times",
plot!(p2, 0:0.1:params[:T], λ *(0:0.1:params[:T]), label = "Poisson fit",
title!(p2, "Attack times (lambda=$lambda)")

p3 = histogram(intervals, bins = 30, normalize = :pdf, label = "Histogram of intervals",
 x_dens = range(0, maximum(intervals), length=100)
 theor_dens = pdf.(Exponential(1/ λ), x_dens)
plot!(p3, x_dens, theor_dens, label = "Histogram of intervals", lw=2,
 title!(p3, "Histogram of intervals (lambda=$lambda)")

p4 = qqplot(Exponential(1/ λ), intervals, qqline = :identity,
 xlabel = "Theoretical quantiles", ylabel = "Sample quantiles",
 title = "QQ-plot (lambda=$lambda)")

combined = plot(p1, p2, p3, p4, layout = (2,2), size = (1000, 800))
plot_filename = joinpath(parametric_plots_dir, "attack_sim_$(λ).
 savefig(combined, plot_filename)
println("Attack simulation results saved to $plot_filename")

```

```

summary[] = Dict(
 :emp_prob => data[:emp_prob],
 :theor_prob => data[:theor_prob],
 :filename => filename
)
println(" = $: = $(data[:theor_prob]), ")
end

```

...

= 2.0 C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\attack\_sim\T=24.0\_num\_hours\_for\_es  
 =2.0 C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\plots\parameter\_sweep\attack\_sim\_ =2.0.p  
 = 2.0: = 8.308224368480666e-6, = 0.0

= 5.0 C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\attack\_sim\T=24.0\_num\_hours\_for\_es  
 =5.0 C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\plots\parameter\_sweep\attack\_sim\_ =5.0.p  
 = 5.0: = 0.013695268598382881, = 0.01  
 = 8.0 C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\attack\_sim\T=24.0\_num\_hours\_for\_es  
 =8.0 C:\Users\Admin\work\study\2026-

```

1\2026-1==study--security-modeling\2026-1--study--security-
modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=8.0.p
λ = 8.0:

| | |
|---------------------|-------|
| 0.18411420744145301 | 0.183 |
|---------------------|-------|

| | |
|---------------------|-------|
| 0.18411420744145301 | 0.183 |
|---------------------|-------|

 λ = 12.0 C:\Users\Admin\work\study\2026-
1\2026-1==study--security-modeling\2026-1--study--security-
modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_es

| | |
|--------------------|-------|
| 0.6527705824458284 | 0.651 |
|--------------------|-------|

| | |
|--------------------|-------|
| 0.6527705824458284 | 0.651 |
|--------------------|-------|

 λ = 15.0 C:\Users\Admin\work\study\2026-
1\2026-1==study--security-modeling\2026-1--study--security-
modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_es

| | |
|--------------------|-------|
| 0.8815355884709849 | 0.884 |
|--------------------|-------|

| | |
|--------------------|-------|
| 0.8815355884709849 | 0.884 |
|--------------------|-------|


```

Сохраняем сводные данные отдельно

```

summary_filename = datadir("parameter_sweep", "summary_λ_values.jld2")
@save summary_filename λ_values summary
println("Сводные данные сохранены в $summary_filename")

```

```


| | |
|---------------------|-------|
| 0.18411420744145301 | 0.183 |
|---------------------|-------|

C:\Users\Admin\work\study\2026-
1\2026-1==study--security-modeling\2026-1--study--security-
modeling\labs\lab02\project\data\parameter_sweep\summary_λ_values.j

```

Построение графика зависимости вероятностей от  $\lambda$

```

xs = [x for x in x_values]
theor_probs = [summary[x][:theor_prob] for x in xs]
emp_probs = [summary[x][:emp_prob] for x in xs]

p = plot(xs, [theor_probs emp_probs],
 label = ["XXXXXXXXXXXXXXXX P(>10)" "XXXXXXXXXXXXXXXX P(>10)"],
 marker = :circle,
 xlabel = "XXXXXXXXXXXXXXXX x (XXXX/XXXX)",
 ylabel = "XXXXXXXXXXXXXXXX P(>10)",
 title = "XXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXX XXXX")
global_plot_path = plotsdir("parameter_sweep.png")
savefig(p, global_plot_path)
println("XXXXXX XXXXXXX XXXXXXXXXXXXXXXXXXX XXXXXXXX x $global_plot_path")

```

XXXXXX XXXXXXX XXXXXXXXXXXXXXXXXXX XXXXXXXX x C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\plots\parameter\_sweep.png

Опционально: таблица в формате CSV

```

df = DataFrame(x = xs, theoretical = theor_probs, empirical = emp_probs)
CSV.write(datadir("parameter_sweep", "summary.csv"), df)
println("XXXXXXXXXX XXXXXXXXXXXXXXX x ", datadir("parameter_sweep", "summary.csv"))

```

XXXXXXXXXX XXXXXXXXXXXXXXX x C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab02\project\data\parameter\_sweep\summary.csv

Подробнее с возможностями использованных инструментов можно ознакомиться в документации [1–3].

## 5 Контрольные вопросы

**1. Какими свойствами обладает простейший поток событий? Почему он часто используется для моделирования атак?**

Простейший (пуассоновский) поток обладает тремя свойствами:

1. **Стационарность** — вероятность появления  $k$  событий за интервал длины  $t$  зависит только от  $t$  и не зависит от момента начала отсчёта.
2. **Отсутствие последствия** — число событий в непересекающихся интервалах независимо.
3. **Ординарность** — вероятность появления двух и более событий за бесконечно малый интервал пренебрежимо мала.

Используется для моделирования атак потому что это математически простой и хорошо изученный процесс с готовыми аналитическими формулами. Согласно теореме Пальма–Хинчина, суперпозиция большого числа независимых стационарных ординарных потоков сходится к пуассоновскому — что соответствует реальной ситуации, когда атаки приходят от множества независимых источников.

**2. Как сгенерировать реализацию пуассоновского потока на интервале времени?**

Используется метод на основе экспоненциального распределения интервалов. Алгоритм:

1. Задать интенсивность  $\lambda$  и длительность  $T$ .
2. Установить  $t = 0$ .

3. Генерировать  $\tau \sim \text{Exp}(\lambda)$ , прибавлять к  $t$ .
4. Если  $t \leq T$  — записать момент события, иначе — остановиться.

### 3. Как проверить, что интервалы между событиями распределены экспоненциально?

Три способа:

- **Гистограмма с теоретической плотностью** — строим гистограмму интервалов и накладываем кривую  $\text{Exp}(\lambda)$ . Если совпадают — распределение экспоненциальное.
- **QQ-plot** — строим квантиль-квантильный график интервалов против теоретического экспоненциального распределения. В нашем коде это `qqplot(Exponential(1/λ), intervals, qqline = :identity)`. Если точки лежат на прямой  $y = x$  — распределение экспоненциальное.
- **Критерий Колмогорова–Смирнова** — если  $p\text{-value} > 0.05$ , нет оснований отвергать гипотезу об экспоненциальности.

### 4. В чём преимущества использования DrWatson для организации вычислительного эксперимента?

- **Стандартная структура проекта** — папки `src/`, `scripts/`, `data/`, `plots/` создаются автоматически.
- **Воспроизводимость** — проект инициализируется как Julia-окружение с `Project.toml` и `Manifest.toml`, фиксируя версии всех пакетов.
- **@quickactivate** — макрос гарантирует что скрипт выполняется в контексте именно этого проекта.
- **savename** — автоматически формирует имя файла из словаря параметров, исключая путаницу между результатами разных запусков.
- **Кэширование** — `produce_or_load` не пересчитывает результаты если файл уже существует.

## 5. Что такое **produce\_or\_load** и как он работает?

`produce_or_load` — функция DrWatson для кэширования результатов вычислений. Логика работы:

1. Формирует путь к файлу на основе параметров через `savename`.
2. Если файл существует — загружает данные с диска, вычисление не запускается.
3. Если файла нет — выполняет вычисление и сохраняет результат.

В нашей лабе реализована та же логика вручную через `isfile + @save/@load`, что эквивалентно `produce_or_load`.

## 6. Какая структура проекта создаётся DrWatson? Для чего нужны папки?

project/

```
├─ data/ # XXXXXXXXXXXX XXXXXXXXXXXX, XXXXXXXXXXXX JLD2/CSV XXXXX
├─ scripts/ # XXXXXXXXXXXX XXXXXXXX
├─ src/ # XXXXXXXX XXX — XXXXXXXX X XXXXXXXX
├─ plots/ # XXXXXXXXXXXX XXXXXXXX
├─ notebooks/ # Jupyter notebooks
├─ markdown/ # Quarto-XXXXXXXXXXXXX
├─ Project.toml # XXXXXX XXXXXXXXXXXXXXX XXXXXXXX
└─ Manifest.toml # XXXXXXXXXXXXXXXXXXXX XXXXXXX XXXX XXXXXXXX
```

`src/` — переиспользуемый код, подключается через `include(scrdir(...))`.  
`scripts/` — точки входа, запускаются напрямую. `data/` — только результаты, не редактируется вручную. `plots/` — только графики, генерируются скриптами.

## 7. Как задать набор параметров для множественных запусков в DrWatson?

Вынести параметры в отдельный файл `scripts/params.jl`:

```
default_params = Dict{
 :X => 5.0,
 :T => 24.0,
 :num_hours_for_est => 10000
}
```

В других скриптах подключать и переопределять нужные поля через `merge`. Для множественных запусков — создавать список словарей и итерировать по нему, как реализовано в `parameter_sweep.jl` через цикл по `X_values`.

## 6 Выводы

1. Сформирована структура рабочего пространства на основе DrWatson, обеспечивающая разделение исходных кодов, данных и документации.
2. Реализована симуляция пуассоновского потока атак двумя способами: почасовые счётчики из распределения Пуассона и точные моменты через экспоненциальные интервалы.
3. Выполнен статистический анализ результатов: гистограммы, накопленный график, QQ-plot — все подтверждают соответствие теоретическим распределениям.
4. Проведено параметрическое исследование: показана зависимость  $P(>10)$  от интенсивности  $\lambda$  — при росте  $\lambda$  от 2 до 15 вероятность возрастает от  $\sim 0$  до  $\sim 0.88$ .
5. Исследована сходимость оценки вероятности редкого события: для надёжной оценки  $P(>10)$  при  $\lambda=5$  необходим объём выборки порядка 10 000–100 000 часов.
6. Выполнена интеграция вычислительных экспериментов с их описанием за счёт преобразования кода в литературный стиль.
7. Автоматизирована генерация артефактов (чистый код, Notebook, отчёт Quarto), что повышает воспроизводимость исследования.

# Список литературы

1. DrWatson.jl Documentation. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> (дата обр. 22.08.2026).
2. JuliaLang. — URL: <https://julialang.org/> (дата обр. 22.08.2026).
3. Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/v2/> (дата обр. 22.08.2026).